

Asymptotic complexity / common orders

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$
Worst-case $\geq$ Average-case $\geq$ Best-case.

Rule of thumb: in polynomial $an^k + \dots$, keep highest power term.

Classic recurrences

$T(n) = T(n-1) + c \Rightarrow T(n) = \Theta(n)$
 $T(n) = T(n-1) + n \Rightarrow T(n) = \Theta(n^2)$
 $T(n) = 2T(n/2) + c \Rightarrow T(n) = \Theta(n)$
 $T(n) = 2T(n/2) + n \Rightarrow T(n) = \Theta(n \log n)$ (mergesort)
 $T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\log n)$ (binary search)
 $T(n) = 2T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$

Master theorem (sketch) $T(n) = aT(n/b) + f(n)$, $a \geq 1, b > 1$. Let $n^{\log_b a}$ be the pivot.

Case 1: $f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow T(n) = \Theta(n^{\log_b a})$.

Case 2: $f(n) = \Theta(n^{\log_b a} \log^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity $\Rightarrow T(n) = \Theta(f(n))$.

Proof templates

Induction: Base ($n = n_0$), IH: assume for n , Step: prove for $n+1$ using IH.

Loop invariant: (1) Init: true before first iteration. (2) Maintain: if true before, still true after one loop. (3) Termination: when loop ends, invariant + loop condition give postcondition.

Greedy exchange argument: Start from an optimal solution, if it differs from greedy at some choice, exchange to make it more greedy but not worse cost; repeat until identical to greedy $\Rightarrow$ greedy is optimal.

Arrays / Lists / Stack / Queue

Array: contiguous, random access $O(1)$, insert/delete middle $O(n)$.

SLL: no random access, insert/delete given prev $O(1)$. DLL: node has (prev,next).

Op	Array	SLL	DLL
Access idx	$O(1)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$
Ins/del head	$O(1)$	$O(1)$	$O(1)$
Ins/del middle*	$O(n)$	$O(1)$	$O(1)$

*given pointer (and prev for SLL).

SLL reverse / queue tricks

SLL reverse *traversal*, no extra space, cannot change links:

for $i = n \rightarrow 1$ scan from head to i -th node $\Rightarrow 1 + 2 + \dots + n = O(n^2)$.

In-place list *reverse* (allowed to change links): one pass with (prev,cur,next) pointers $\Rightarrow O(n)$ time, $O(1)$ space.

Stack via SLL (head=top): push/pop/top all $O(1)$.

Queue via SLL (head=front, tail=rear): enqueue/dequeue/front all $O(1)$.

Queue with avg in $O(1)$: maintain sum, cnt.

enqueue(x): push, sum+= x , cnt++; dequeue(): pop x , sum-= x , cnt--;

getAverage(): sum / cnt. Invariant: sum= $\sum$ elements.

Queue by two stacks (amortized):

in-stack for enqueue, out-stack for dequeue.

enqueue(x): push to in. dequeue(): if out empty, move all from in to out, then pop out.

Each element moved in $\rightarrow$ out at most once $\Rightarrow$ amortized $O(1)$ per op.

Trees / BST / Balanced BST / Map

Tree: connected acyclic graph. Rooted tree: distinguished root, parent/child/sibling, depth, height.

BST property: for every node x : all keys in left subtree $< key(x)$, right subtree $> key(x)$.

Height h : average $O(\log n)$ for balanced trees, worst-case $O(n)$ (degenerate).

BST operations (height h): search, insert, delete all $O(h)$.

Inorder traversal yields sorted keys.

Range query $[a, b]$: traverse only relevant subtrees, complexity $O(\log n + k)$ where $k = \#$ reported keys.

BST deletion: case 0/1 child: splice node out. Case 2 children: replace by inorder successor (min in right subtree) or predecessor, then delete that node (now has at most 1 child).

Computing subtree sizes: Use a post-order DFS; the subtree size of a node is $1 + \sum$ of its children's sizes, giving a linear-time bottom-up computation.

Visiting all nodes on a given level k : Carry the current depth as a parameter during DFS; visit a node only when its depth equals k , still $O(n)$ overall.

Finding the next node in pre-order: If a node has a left child, the next is its left; otherwise its right; otherwise climb upward until reaching an ancestor whose right subtree has not been visited, which becomes the successor.

Finding the next node in in-order: If the node has a right child, the successor is the leftmost node in that right subtree; otherwise climb upward until reaching an ancestor where the current node lies in its left subtree.

Finding the next node in post-order: If the node is the root, no successor exists; otherwise, if it is the right child (or the parent has no right child), the parent is next; if it is the left child and the parent has a right subtree, descend into that right subtree to find its first post-order node.

Computing height or balance factors: Perform a post-order DFS; return each node's height as $1 + \max(h_L, h_R)$ and compute the balance factor as $|h_L - h_R|$, all in $O(n)$.

Euler tour without recursion: Model traversal as a state machine (u, W) where $W \in \{L, B, R\}$ indicates which side of node u the tour is currently on; update the state in $O(1)$ per step for a total of $O(n)$.

Computing the tree's diameter: Use one DFS to compute left and right subtree heights at every node; the longest simple path is the maximum value of $h_L(u) + h_R(u) + 2$ across all nodes.

BST local restructuring (rotations): For any ordered triple $x < y < z$, rebuild their local subtree into shape y as root with children x, z and reattach the four subtrees T_0, T_1, T_2, T_3 to preserve BST order; this fixes local imbalance with $O(1)$ pointer changes.

Height vs. number of nodes: A binary tree of height h has at most $2^h - 1$ internal nodes; therefore any BST with n keys must satisfy $\log_2(n) < h$, i.e. height is at least logarithmic in n .

External nodes count: In the external-node view of BSTs, every tree with n internal nodes contains exactly $n+1$ external nodes; prove by induction by contracting and restoring a leaf node.

Correct BST validation: Checking only parent-child ordering is insufficient; proper verification carries an allowable key range $[min, max]$ down the recursion and requires each node's key to lie in its inherited interval to maintain global BST ordering.

Finding maximum / minimum: Maximum is reached by repeatedly following right pointers until a node with no right child; minimum similarly by following left pointers; both take $O(h)$.

Finding the second largest key: Let w be the maximum. If w has a left subtree, the second largest is the maximum in $w.left$; otherwise it is w 's parent (unless w is the only node).

Order-statistic (k-th smallest): Augment each node with `size` = number of nodes in its subtree. To find the k -th smallest, compare k with `left.size`: go left if $k \leq \text{left.size}$, return the current node if $k = \text{left.size} + 1$, otherwise go right with $k \leftarrow k - \text{left.size} - 1$, all in $O(h)$.

Removing all keys in interval $[k_1, k_2]$: Use a range search to reach a node whose key lies in $[k_1, k_2]$; prune entire subtrees that are fully $< k_1$ or fully $> k_2$, and delete only nodes inside the interval. The total work is $O(h + s)$ where s is the number of removed keys.

AVL tree: height-balanced BST. For each node, $|height(left) - height(right)| \leq 1$. Height $O(\log n)$.

Ops: search/insert/delete $\Rightarrow O(\log n)$.

Rebalancing via rotations: after local height violation.

Tree traversals (DFS)

Pre-order: visit root, then left subtree, then right subtree (root-left-right).

In-order: visit left subtree, then root, then right subtree (left-root-right).

Post-order: visit left subtree, then right subtree, then root (left-right-root).

Rotations:

– LL: heavy on left-left $\Rightarrow$ single right rotation.

– RR: heavy on right-right $\Rightarrow$ single left rotation.

– LR: heavy on left-right $\Rightarrow$ left rotation on left child then right rotation.

– RL: heavy on right-left $\Rightarrow$ right rotation on right child then left rotation.

Map ADT

Ops: get/put/remove/containsKey/size/iterate.

Implementations: array of (key,val), sorted array, BST, AVL, hash table.

Structure	Avg get/put/rem	Worst
Unsorted array/list	$O(n)$	$O(n)$
Sorted array	$O(\log n)$ (search) + shift	$O(n)$
BST (random)	$O(\log n)$	$O(n)$
AVL	$O(\log n)$	$O(\log n)$
Hash table	$O(1)$ exp.	$O(n)$

Priority queue / Binary heap

Stores key-value items (k, v) ; only the smallest key can be removed. Ops: insert, min, remove_min, size, is_empty.

Implementation	insert	min	remove_min	size	is_empty
Unsorted array/list	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Sorted array/list	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Binary min-heap	$O(\log n)$	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$

Binary min-heap: complete binary tree + heap-order (key(parent) $\leq$ keys(children)); array layout: root at 1, children of i at $2i, 2i+1$.

Other operations on min-heap (size n): search for given key / membership $O(n)$; find maximum key $O(n)$ (no ordering among leaves).

Heap operations (height $h = \Theta(\log n)$): find-min $O(1)$; insert = add at end + bubble-up $O(h)$; delete-min = swap root with last + remove last + bubble-down $O(h)$; build-heap from n items by bottom-up heapify $O(n)$ (not $O(n \log n)$).

Heapsort via heap: build-heap $O(n) + n$ delete-min, each $O(\log n) \Rightarrow$ worst-case $\Theta(n \log n)$ (need $\Omega(n)$ deletes and each delete touches height $\Theta(\log n)$).

Sorting via priority queues

Algorithm	PQ representation	per-op costs	Total time
Selection sort	Unsorted list	insert $O(1)$, remove_min $O(n)$	$\Theta(n^2)$
Insertion sort	Sorted list	insert $O(n)$, remove_min $O(1)$	$\Theta(n^2)$
Heap sort	Binary heap	insert / remove_min $O(\log n)$	$\Theta(n \log n)$

Selection sort: inner loop always scans full suffix $\Rightarrow$ always $\Theta(n^2)$.

Insertion sort: running time $O(n + I)$ where I is the number of inversions; each shift removes one inversion, worst-case $I = \Theta(n^2)$.

Heap-based patterns

k -th largest (or smallest) in unsorted array of n items: maintain size- k heap; insert first k , then for each new item if it improves the heap, pop top and insert; total time $O(n \log k)$. Merge k sorted lists (total N items): min-heap of size k on current heads; repeatedly extract-min and push its successor; time $O(N \log k)$.

Analysis tips

When analysing claimed $O(n)$ algorithms using heaps/PQs, remember each heap delete/insert is $\Omega(\log n)$ in comparison model, and array-based structures cannot support both $O(\log n)$ search and $O(1)$ insertion unless further structure (e.g. balanced BST) is specified.

Graph basics

Graph $G = (V, E)$, $|V| = n$, $|E| = m$. Directed/undirected.

Degree, indegree/outdegree. Path, simple path, cycle.

Connected (undirected): path between every pair.

DAG: directed acyclic graph.

Representations:

Rep	Space	Edge iteration
Adjacency matrix	$O(n^2)$	scan row $O(n)$
Adjacency list	$O(n + m)$	$O(\deg(v))$ per vertex
Edge list	$O(m)$	scanning edges $O(m)$

Adj matrix good when graph dense or constant-time edge existence check needed. Adj list good when sparse for $O(n + m)$ traversals.

BFS / DFS

BFS (queue, unweighted shortest path):

Init: mark all unvisited, set distance[root]=0. Enqueue root.

While queue not empty: dequeue u , for each neighbor v if unvisited, set dist[v]=dist[u]+1, parent[v]=u, enqueue v .

Complexity BFS: $O(n + m)$ on adjacency list.

DFS (stack/recursion):

For each vertex u unvisited: DFS-Visit(u).

DFS-Visit(u): mark visited; for each neighbor v : if unvisited, parent[v]=u, recursively DFS-Visit(v).

Complexity DFS: $O(n + m)$ on adjacency list.

Bipartite (undirected): V partitioned into A, B , $E \subseteq A \times B$; equivalently, graph is 2-colorable; equivalently, no odd cycles.

Test bipartiteness in $O(n + m)$: for each connected component, BFS and 2-color vertices by layers; if any edge connects same-layer (same-color) vertices is not bipartite; otherwise bipartite. Applications:

– BFS: shortest path (unweighted), bipartite testing, layer decomposition.

– DFS: reachable vertices, topological sort (DAG), cycle detection, cut vertices/bridges, strongly connected components.

BFS layers & mandatory vertex

In an undirected graph, BFS from s gives layers $L_0, \dots, L_k$ with $L_i = \{v : \text{dist}(s, v) = i\}$. If dist(s, t) = $k > n/2$, some middle layer L_i ($0 < i < k$) must contain exactly one vertex u (pigeonhole principle). Every $s-t$ path must pass through u (otherwise we could build an $s-t$ path avoiding u , contradicting the layer-size argument), so u is a mandatory vertex for all $s-t$ paths.

Get-stuck vertex via adjacency matrix

A get-stuck vertex v has out-degree 0 and in-degree $n-1$ (its row in the adjacency matrix is all 0, its column is all 1 except the diagonal). On the $n \times n$ adjacency matrix, start at $(1, 1)$ and move only right or down according to the entries; this walk leaves the matrix after at most $2n-1$ steps and identifies a unique candidate row. Then check that row and column in $O(n)$ time to verify the get-stuck conditions, for total time $O(n)$.

Component minimum label (small(i))

For each vertex i , define small(i) as the minimum-labeled vertex reachable from i . Run one DFS/BFS to find all connected components in $O(n + m)$; for each component C , let $a = \min C$ and set small(v) = a for all $v \in C$. Overall complexity is $O(n + m)$.

Cut vertices (articulation points)

Run DFS, record depth/level lvl[u] and for each vertex u compute low[u], the minimum level reachable from u by following zero or more tree edges downward and then at most one back edge upward (this is the same as the “down.and.up” value).

Characterisation: the DFS root is a cut vertex iff it has at least two DFS children; a non-root u is a cut vertex if it has a child v with low[v] $\geq$ lvl[u] (the subtree of v cannot reach any ancestor of u without using u). All articulation points can be found in $O(n + m)$.

Tree DP: subtree minimum

In a rooted tree, each vertex u has a value $A[u]$. We want $B[u] = \min\{A[v] : v \in T_u\}$ where T_u is the subtree rooted at u . Use a post-order DFS: first recurse on children to get their $B[v]$, then set $B[u] = \min\{A[u], \{B[v] \mid v \text{ child of } u\}\}$. Each edge is processed a constant number of times, so the total running time is $O(n)$.

Cycle detection in undirected graphs

Run DFS and record parent[u] and visited[u]. When exploring an edge (u, v) :

– if v is unvisited, set parent[v] = u and recurse;

– else if $v \neq \text{parent}[u]$, we have found a cycle: follow parent[] pointers from u back to v to output the cycle.

Before the first back-edge is found, the DFS tree has at most $n-1$ edges, so the work is $O(n)$ up to discovering the first cycle; in general traversal cost is $O(n + m)$.

Cut edges (bridges)

In the same DFS, for each tree edge (u, v) with parent u and child v , use low to test whether it is a bridge: if low[v] $>$ lvl[u] then (u, v) is a cut edge (no back edge from the subtree of v reaches u or any ancestor of u); otherwise it is not a bridge. All bridges can be listed in $O(n + m)$ time.

Topological sort (DAG): either DFS + postorder (reverse finishing times) or repeatedly remove in-degree-0 vertices (Kahn).

Cut edge / cut vertex (ideas)

Bridge (cut edge): edge whose removal increases $\#$ connected components.

Using DFS discovery time $disc[u]$ and low-link $low[u]$:

– Tree edge (u, v) is bridge iff $low[v] > disc[u]$.

Articulation point (cut vertex): root with ≥ 2 DFS children or non-root u with some child v s.t. $low[v] \geq disc[u]$.

Shortest paths / Dijkstra

Single-source shortest paths (SSSP) in weighted graph. Dijkstra assumes all edge weights ≥ 0 .

Invariant: when vertex removed from PQ/visited, its distance is final shortest distance.

Dijkstra algorithm (sketch):

Init: $dist[s] = 0$, $dist[v] = \infty$ for $v \neq s$. PQ keyed by dist.

While PQ not empty: extract-min u ; for each edge (u, v) with weight w : if $dist[u] + w < dist[v]$, relax: set $dist[v] = dist[u] + w$, update PQ. Complexity (using adjacency list):

– With array PQ: $O(n^2)$ (dense graphs ok).

– With binary heap: $O((n + m) \log n) \approx O(m \log n)$.

Works only for non-negative weights. For negative edges, use Bellman-Ford ($O(nm)$) or other algorithms (not usually core in basic DS/Algo).

MST: Kruskal / Prim

Minimum spanning tree (MST, undirected, connected, non-negative weights).

Cut property: for any cut, the minimum-weight edge crossing the cut that is safe (connecting different components) belongs to some MST.

Cycle property: in any cycle, the maximum-weight edge is not in some MST.

Kruskal's algorithm (greedy on edges):

1. Sort all edges by nondecreasing weight.

2. Init forest with all vertices, no edges. Use Union-Find for components.

3. For each edge (u, v) in increasing weight order: if u, v in different components, add edge, union sets.

Stop when $n-1$ edges chosen.

Complexity: sorting $O(m \log m)$, Union-Find almost $O(m \alpha(n))$, so $O(m \log m)$.

Prim's algorithm (greedy on frontier):

Maintain a growing tree starting from arbitrary root.

At each step, pick minimum-weight edge (u, v) where u in tree, v outside.

Implementation w/ binary heap and adjacency list: $O(m \log n)$.

Dijkstra (directed extension): Relax only outgoing edges (u, v) from the PQ-extracted vertex u ; same algorithm and $O((n + m) \log n)$ complexity as undirected. **Shortest path**

under weight transforms: Multiplying all weights by $\alpha > 0$ preserves shortest paths; adding constants or squaring weights may change SP but do not affect MST (edge order unchanged).

Tie-breaking shortest path (min edges after min weight): Use modified weights $\ell''(e) = M\ell(e) + 1$ with $M > |V|$ so total weight dominates, edges break ties; run standard Dijkstra.

Shortest path with vertex + edge costs: Use node-splitting: each u becomes (u_{in}, u_{out}) with cost edge $w(u)$, and every original edge (u, v) becomes (u_{out}, v_{in}) ; run Dijkstra on the transformed graph.

Improving-MST algorithm: Start with any spanning tree; for each non-tree edge, add it to form a cycle and delete the cycle's heaviest edge. Correct by cut/cycle properties; runs in $O(nm)$.

Reverse-MST algorithm: Sort edges in decreasing order; repeatedly delete the current heaviest edge if graph stays connected. Uses cut property; complexity $O(m^2)$.

Max-bandwidth (maximum bottleneck) paths: Compute a Maximum Spanning Tree; for any u, v , the minimum edge on their unique MST path achieves the maximum possible bottleneck between u, v .

MST: Cut & Cycle properties

Cycle property: in any cycle, the *maximum-weight* edge is never in *any* MST.

Cut property: for any cut, the *minimum-weight* edge crossing that cut must be in *every* MST.

Unit-weight graphs

Even if all edges have weight 1, not all s - t paths are shortest; only those with minimum hops.

Dijkstra

Requires all weights ≥ 0 ; fails with negative-weight edges.

Directed extension: relax only outgoing edges (u, v) .

Shortest-path tree (SPT)

SPT depends on root; different from MST.

For root s , SPT contains for each v its parent on the shortest $s \rightarrow v$ path found by Dijkstra/BFS.

Common MST/SPT confusions

MST minimizes total weight, not distances from a root.

SPT minimizes root-to-vertex distances, not total weight.

MST edges $\neq$ SPT edges in general.

Union-Find (Disjoint sets)

Ops: MakeSet(x), Find(x) (rep element), Union(x, y).

Implementation: parent array, optionally rank/size.

Path compression (during Find) + union by rank/size $\Rightarrow$ amortized $O(\alpha(n))$ per op, where α is inverse Ackermann (practically ≤ 4).

Greedy algorithms

General pattern: make locally best choice according to some criterion; prove global optimality via exchange argument or structural property.

Interval scheduling (max number of non-overlapping intervals):

Greedy rule: pick interval with earliest finish time among remaining compatible intervals.

Correctness: exchange argument (replace opt interval with earlier-finishing one).

Complexity: sort by finish time $O(n \log n)$ + scan $O(n)$.

Interval partitioning (min # classrooms):

Greedy rule: process intervals sorted by start time; assign to lowest-index free room; else open a new room.

Property: #rooms used = maximum depth = clique number ω of interval graph; interval graphs are perfect $\Rightarrow \chi = \omega$.

Complexity: $O(n \log n)$ using min-heap of finishing times.

Interval graph distinctions:

Scheduling = max independent set (choose non-overlapping).

Partitioning = graph coloring (assign different rooms to overlapping).

Fractional knapsack:

Items with value v_i , weight w_i ; fractions allowed.

Greedy rule: sort by density v_i/w_i descending, take greedily.

Correctness: exchange argument. Complexity $O(n \log n)$.

0/1 knapsack: greedy fails (counterexamples exist).

Huffman coding:

Build optimal prefix-free code minimizing expected length.

Greedy: repeatedly merge two least frequent symbols; use min-heap.

Correctness: two least frequent leaves are siblings; can transform any optimal tree to greedy one (exchange/induction).

Complexity $O(n \log n)$.

Greedy algorithms are *not* always optimal, and a problem can have many different greedy strategies; only some of them are correct, and this must be proved.

Huffman coding facts

In any optimal Huffman tree, more frequent characters never have deeper codes than less frequent ones (higher frequency $\Rightarrow$ depth closer to root).

Code length for a string = sum of depths of its characters in the Huffman tree (one bit per edge on the root-to-leaf path).

Greedy proof templates:

Exchange argument: transform any optimal solution into greedy one w/o hurting cost.

Structural property: interval graphs are perfect; earliest finish time makes future choice maximal.

Cut/cycle properties (MST): min cut edge must be in MST; max cycle edge not in MST.

Greedy fails when:

0/1 knapsack, negative edges in shortest path (Dijkstra fails), general graph coloring (non-interval graphs).

Divide & Conquer / Recurrences

Mergesort merge step: merge two sorted length- n arrays with two pointers; each element copied once $\Rightarrow O(n)$ time. Precondition: each subarray is individually sorted (no ordering relation required between the two subarrays). Pattern: split problem into smaller subproblems, solve recursively, then combine.

Typical examples:

Binary search: divide in half, solve one half. $T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$.

Merge sort: split array into two halves, recursively sort, merge. $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$.

Quick sort (average): partition around pivot, recursively sort two sides; average $T(n) = 2T(n/2) + O(n)$; worst-case $T(n) = T(n-1) + O(n) = O(n^2)$.

Matrix multiply (naive): 8 subproblems size $n/2$, $T(n) = 8T(n/2) + O(n^2) \Rightarrow \Theta(n^3)$.

Matrix multiply (Strassen idea): 7 subproblems size $n/2$, $T(n) = 7T(n/2) + O(n^2) \Rightarrow$

$\Theta(n^{\log_2 7})$.

New-sort (toy D&C sort): recursively sort $[0, 2n/3]$, then $[n/3, n]$, then $[0, 2n/3]$;

$T(n) = 3T(2n/3) + O(1) \Rightarrow \Theta(n^{\log_{3/2} 3}) \approx n^{2.71}$.

Local minimum in array: compare mid with neighbors, recurse on strictly-decreasing side;

$T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$.

k -th smallest in two sorted arrays: shrink & conquer using ranks / binary search on indices; discard constant fraction each step $\Rightarrow O(\log(m+n))$.

Typical recurrence solutions (by Master theorem / expansion):

$T(n) = 2T(n/2) + n$ $\Theta(n \log n)$

$T(n) = 2T(n/2) + n \log n$ $\Theta(n \log^2 n)$

$T(n) = 2T(n/2) + n^2$ $\Theta(n^2)$

$T(n) = T(n/2) + n$ $\Theta(n)$

$T(n) = 3T(n/2) + n$ $\Theta(n^{\log_2 3})$

$T(n) = 4T(n/2) + n^2$ $\Theta(n^2 \log n)$

$T(n) = 3T(2n/3) + 1$ $\Theta(n^{\log_{3/2} 3})$

$T(n) = 3T(n/3) + \sqrt{n}$ $\Theta(n)$

$T(n) = T(n/2) + 2^n$ $\Theta(2^n)$

$T(n) = T(n-1) + 1$ $\Theta(n)$

$T(n) = T(n-1) + n$ $\Theta(n^2)$

Extra patterns: $T(n) = aT(n/b) + f(n)$

– If $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$ (borderline).

– Examples: $T(n) = 3T(n/3) + n \Rightarrow \Theta(n \log n)$, $T(n) = 3T(n/3) + \sqrt{n} \Rightarrow \Theta(n)$.

Recurrence pattern recognition

Translate verbal D&C descriptions into $T(n) = aT(n/b) + f(n)$:

– “reduce data size by one, solve recursively, then constant work” $\Rightarrow T(n) = T(n-1) + O(1)$.

– “reduce data size by one, solve recursively, then linear work” $\Rightarrow T(n) = T(n-1) + O(n)$.

– “divide data in half, choose *one* half to solve recursively, constant work to adjust” $\Rightarrow T(n) = T(n/2) + O(1)$.

– “divide data in half, choose *one* half to solve recursively, linear work to adjust” $\Rightarrow T(n) = T(n/2) + O(n)$.

– “divide data in half, solve *both* halves recursively, constant work to combine” $\Rightarrow T(n) = 2T(n/2) + O(1)$.

– “divide data in half, solve *both* halves recursively, linear work to combine” $\Rightarrow T(n) = 2T(n/2) + O(n)$ (e.g. mergesort).

Rules of thumb: #recursive calls $\Rightarrow a$; shrink factor of input $\Rightarrow b$; description of extra work (constant/linear) $\Rightarrow f(n)$.

Algorithm correctness templates

Proving loop correctness:

1. Choose invariant I capturing “what is already correct” at loop start.

2. Prove init: before first iteration, I holds.

3. Prove maintain: assuming I before an iteration, after executing body once I still holds.

4. Prove termination: when loop ends (condition false), invariant I + negated condition $\Rightarrow$ postcondition.

Proving divide & conquer correctness:

Use strong induction on input size n . Assume recursive calls work for smaller inputs per spec; show combining step produces correct result for size n . For selection / local-minimum style D&C, The key is to prove that the side chosen recursively must contain a solution.

Proving complexity:

– For simple loops: count iterations $\times$ cost per iteration.

– For nested loops: sum over indices or multiply when independent.

– For recursive algorithms: set up recurrence and solve via substitution, recursion tree, or Master theorem.

Greedy correctness: structure typical proof as:

1. Show greedy choice is safe: there exists some optimal solution including this choice.

2. After making greedy choice, remaining subproblem is smaller instance of same type.

3. Argue by induction on problem size or number of choices.

Data structure invariants:

– BST: in-order yields sorted keys.

– Heap: parent key $\leq$ children.

– AVL: balance factor $\in \{-1, 0, 1\}$ at every node.

– Hash table: each element is in bucket index $h(key)$ (or in its probe sequence).

Map ADT and hashing

Map: collection of key-value pairs, at most one item per key.

Core ops: get(k), put(k, v), remove(k), size(), isEmpty().

Map implementations (time for n items, table size N)

Implementation	get	put	remove
Unsorted list (linked)	$O(n)$	$O(1)$ new / $O(n)$ update	$O(n)$
Direct-address array (keys $0..N-1$)	$O(1)$	$O(1)$	$O(1)$
Hash table + chaining (expected)	$O(1 + \alpha)$	$O(1 + \alpha)$	$O(1 + \alpha)$
Hash table + chaining (worst)	$O(n)$	$O(n)$	$O(n)$
Hash table + linear probing (expected)	$O\left(\frac{1}{1-\alpha}\right)$	$O\left(\frac{1}{1-\alpha}\right)$	$O\left(\frac{1}{1-\alpha}\right)$
Hash table + linear probing (worst)	$O(n)$	$O(n)$	$O(n)$
Cuckoo hashing	$O(1)$	expected $O(1)$	$O(1)$

Notes: $\alpha = n/N$ is load factor. Direct addressing needs dense integer key range and $O(N)$ space. Chaining uses a linked list per bucket. Linear probing is open addressing with a probe sequence; deletions use a special DEFUNCT marker. Cuckoo hashing uses 2 tables and 2 hash functions.

Hash functions

Hash function $h : K \rightarrow \{0, \dots, N-1\}$, usually $h(x) = h_2(h_1(x))$ where:

– h_1 (hash code): map key to an integer (e.g. polynomial hash for strings).

– h_2 (compression): map integer to $[0, N-1]$, e.g. $h_2(z) = z \bmod N$ with prime N .

Goals: easy to compute; spread keys roughly uniformly to keep collisions rare.

Collision handling

Separate chaining: each table cell stores a (short) linked list of entries. Expected list length $\approx \alpha \Rightarrow$ expected $O(1 + \alpha)$ per operation.

Linear probing (open addressing): if $A[h(k)]$ is full, try $h(k) + 1, h(k) + 2, \dots$ (wrap around) until key/empty/DEFUNCT found; expected probes $\approx 1/(1 - \alpha)$ when hash values behave randomly.

Cuckoo hashing: two tables T_1, T_2 with h_1, h_2 ; each key sits in exactly one of $T_1[h_1(k)], T_2[h_2(k)]$; lookup checks at most two cells; insertion may trigger evictions and occasional relhash.

Load factor & resizing

Load factor $\alpha = n/N$. Keep α bounded away from 1 (e.g. $\alpha < 0.7$). When α too large, allocate a larger table (often doubling N) and reinsert all keys. With geometric resizing, rehashing cost is amortized $O(1)$ per operation.

Set / Multiset via hashing

Set ADT: store each element x as a key in a hash Map (value can be true or dummy); add/contains/remove all expected $O(1)$.

Multiset (bag): store element as key and its count as value; add increments count, remove decrements, count(e) returns frequency.

Typical hashing patterns

Set via hash table: store each element x as key in a hash Map with dummy value; operations add(x), remove(x), contains(x) all expected $O(1)$; space $O(n)$.

Multiset / bag via hashing: use hash Map key = element, value = count. add(x): if x not present put $(x, 1)$, else increment count.

remove(x): decrement count and delete key when count reaches 0. This supports frequency queries and iteration over distinct elements in expected $O(1)$ per op.

Multimap via hashing: key maps to a linked list (or dynamic array) of values. put(k, v): append v to the list stored at key k .

get(k): returns the whole list; time $O(1 + s)$ where s is the list size. Useful when many items share the same key.

Mode / most frequent element in array: scan array once while maintaining a hash Map from value to count (multiset pattern); track the current maximum count during updates. Under expected $O(1)$ hash ops and constant load factor, total time $O(n)$ and space $O(n)$.

Birthday / collision detection with small key range: if keys lie in a fixed small domain (e.g. 365 birthdays), keep a hash set (or boolean array) of seen keys; for each new item, check membership then insert; the first duplicate can be found in expected $O(1)$ per item, overall $O(n)$ and even $O(1)$ when n is bounded by the domain size.

k -gram (substring / n -gram) statistics: slide a window of length k over a sequence of words or characters; treat each k -gram as a key in a hash Map storing its frequency.

Each step updates the count for one k -gram $\Rightarrow$ total expected time $O(n)$ for sequence length n .

Cuckoo hashing insertion failures: during insertion repeatedly evict conflicting items between the two tables; if the number of evictions exceeds a constant multiple of the table size (or a visited-position flag is hit), declare a cycle and rebuild the tables with new hash functions.

This keeps expected insertion time $O(1)$ while guaranteeing termination via occasional relhash.